

Informe tecnico - Lavanderia Exclusiva

Version tecnica para programadores

Lavanderia Exclusiva

Documento preparado para explicar el funcionamiento, la tecnologia, la base de datos, las conexiones y el despliegue del sistema.

Contenido

- 1. Identificación del proyecto
- 2. Stack técnico
- 3. Arquitectura general
- 4. Rutas y permisos
- 5. Conexión de base de datos
- 6. Despliegue
- 7. Base de datos y tablas principales
- 8. Modelos principales
- 9. Servicios de dominio
- 10. Flujos funcionales
- 11. Seguridad técnica
- 12. Integraciones
- 13. Pruebas y verificación
- 14. Comandos útiles
- 15. Resumen técnico para exposición

1. Identificacion del proyecto

Lavanderia Exclusiva es una aplicacion web monolitica construida con Laravel 12. Usa arquitectura MVC clasica: rutas web, controladores HTTP, modelos Eloquent, migraciones, seeders, vistas Blade y servicios de dominio.

Repositorio remoto:

```
https://github.com/CyWolf07/lavanderia-4.0.git
```

Rama de produccion actual:

```
main
```

Ultimo commit desplegado durante la revision:

```
9d0552d feat: validar produccion manual y sincronizar prendas
```

2. Stack tecnico

Capa	Tecnologia
Backend	PHP 8.2
Framework	Laravel 12
ORM	Eloquent
Frontend server-rendered	Blade
CSS	Tailwind CSS
Bundler	Vite
JavaScript	ES modules, Alpine disponible por dependencia
Base de datos	PostgreSQL
Base online recomendada	Supabase PostgreSQL
Hosting	Render Free
Runtime	Docker con Apache
Testing	Pest / PHPUnit

Dependencias principales:

- laravel/framework
- laravel/tinker
- pestphp/pest

- pestphp/pest-plugin-laravel
- laravel/breeze
- tailwindcss
- vite
- laravel-vite-plugin

3. Arquitectura general

La aplicacion sigue el flujo:

```

Navegador
-> Rutas Laravel (routes/web.php)
-> Middleware de autenticacion, estado y roles
-> Controladores
-> Servicios de dominio cuando aplica
-> Modelos Eloquent
-> PostgreSQL
-> Vistas Blade
-> Respuesta HTML

```

Los archivos clave son:

Archivo o carpeta	Funcion
routes/web.php	Define las rutas web y grupos por rol.
app/Http/Controllers	Controladores de admin, produccion, recolector, clientes, PQRS y autenticacion.
app/Models	Modelos Eloquent y relaciones.
app/Services	Logica de negocio reutilizable.
database/migrations	Estructura evolutiva de base de datos.
database/seeders	Datos iniciales: roles, usuarios, catalogos.
resources/views	Pantallas Blade.
Dockerfile	Imagen de produccion.
docker/start-container.sh	Arranque, migraciones, seeders, cache y Apache.
render.yaml	Configuracion de Render.
config/database.php	Conexion dinamica PostgreSQL/SQLite segun variables.

4. Rutas y permisos

Las rutas se organizan por grupos:

Grupo	Middleware	Usuarios
Publico	Ninguno o throttle especifico	Welcome, codigo empresarial, login, registro inicial, password reset.
Protegido	auth, activo	Perfil, dashboard redirect, PQRS, produccion y recolector segun rol.
Admin	auth, activo, rol:admin, programador, throttle.admin	Panel administrativo completo.

Roles definidos:

- `usuario`: registra produccion propia.
- `recolector`: gestiona clientes visibles, facturas, prendas recibidas, gastos y estado de facturas propias.
- `admin`: administra usuarios, clientes, prendas, produccion, facturas, reportes, gastos, pagos e incongruencias.
- `programador`: rol administrativo con acceso a funciones tecnicas como codigo empresarial y dispositivos bloqueados.

El metodo central de resolucion de rol esta en `App\Models\User::tieneRol()`. Soporta rol por columna `rol` y por relacion `rol_id`.

5. Conexion de base de datos

La configuracion esta en `config/database.php`.

Regla importante:

```
'default' => env('DB_CONNECTION') ? ($databaseUrl || $postgresHost ? 'pgsql' : 'sqlite')
```

Esto significa:

- Si existe `DB_CONNECTION`, se usa ese valor.
- Si existe `DATABASE_URL`, `DB_URL` o `PGHOST`, la app usa PostgreSQL.
- Si no hay variables externas, puede caer a SQLite para entornos simples.

La conexion PostgreSQL lee:

- `DATABASE_URL` o `DB_URL`
- `DB_HOST` o `PGHOST`
- `DB_PORT` o `PGPORT`
- `DB_DATABASE` o `PGDATABASE`
- `DB_USERNAME` o `PGUSER`
- `DB_PASSWORD` o `PGPASSWORD`

- DB_SCHEMA o PGSCHEMA
- DB_SSLMODE o PGSSLMODE

En produccion, render .yaml fija:

```
DB_CONNECTION=pgsql
DB_SSLMODE=require
PGSSLMODE=require
DB_SCHEMA=public
DATABASE_URL=<privado>
```

6. Despliegue

El despliegue online usa Render con Docker.

render .yaml define:

```
runtime: docker
plan: free
dockerfilePath: ./Dockerfile
healthCheckPath: /up
autoDeploy: true
```

Render construye la imagen desde el Dockerfile. La imagen final usa:

- PHP 8.2 con Apache.
- Extensiones pdo, pdo_pgsql, bcmath, intl.
- Cliente PostgreSQL 16.
- Build frontend copiado desde etapa Node.
- Vendor PHP copiado desde etapa Composer.

El comando final del contenedor es:

```
start-container
```

El script docker/start-container.sh realiza:

1. Preparacion de entorno.
2. Validacion de variables.
3. Limpieza de caches.
4. php artisan migrate --force.
5. php artisan db:seed --force.
6. Cache de configuracion, rutas y vistas.
7. storage:link si aplica.
8. Inicio de Apache.

7. Base de datos y tablas principales

La base es PostgreSQL. Las tablas principales son:

Tabla	Proposito
roles	Catalogo de roles.
users	Usuarios, credenciales, cedula, contacto, rol, estado y permisos de precio.
clientes	Clientes, numero interno, celular, direccion, barrio, recolector asignado y coordenadas.
prendas	Catalogo de prendas para produccion/lavadero.
recolector_prendas	Catalogo de prendas usadas por recolectores.
prenda_equivalencias	Mapea prenda de recolector contra prenda de produccion cuando los nombres difieren.
facturas_recolector	Ordenes/facturas creadas por recolectores.
factura_recolector_detalle	Lineas de factura: prenda, cantidad, color, valor, subtotal y trazabilidad de lavado.
producciones	Registros de produccion por usuario, prenda, cantidad, total y validacion.
incongruencias_produccion	Diferencias entre lo recibido y lo reportado por lavaderos.
incongruencias_recolector	Incidencias detectadas en facturas de recolector.
historial_producciones	Produccion cerrada por periodo/quincena.
gastos	Gastos por usuario y quincena.
pagos_recolector	Comision calculada para recolectores por quincena.
system_settings	Configuraciones internas como codigo empresarial y modo de produccion.
enterprise_access_controls	Control de dispositivos e intentos fallidos.
audit_events	Bitacora generica de eventos auditables.
sessions, cache, jobs, notifications	Infraestructura Laravel.

8. Modelos principales

Modelo	Responsabilidad
User	Usuario autenticable, roles, estado, permisos y relaciones.
Cliente	Cliente, numeracion automatica y visibilidad por recolector.
Prenda	Prendas para produccion y scope de activas.

Modelo	Responsabilidad
RecolectorPrenda	Prendas para recoleccion y catalogo activo.
FacturaRecolector	Encabezado de orden de recolector, estados y scopes de quincena.
FacturaRecolectorDetalle	Detalle de prendas, colores y trazabilidad de lavado.
Produccion	Produccion diaria, validacion y scope pagables.
HistorialProduccion	Produccion cerrada por periodo.
Gasto	Gastos y calculo de periodo/quincena.
PagoRecolector	Calculo de comision por recolector.
IncongruenciaProduccion	Incidencias de produccion manual.
IncongruenciaRecolector	Incidencias de recoleccion.
SystemSetting	Configuracion clave/valor.
AuditEvent	Registro auditable generico.

9. Servicios de dominio

Servicio	Funcion
ProduccionValidationService	Recalcula produccion por fecha, compara recibido vs reportado y crea incongruencias.
PrendasLavanderoSyncService	Sincroniza catalogo de prendas de lavandero desde prendas de recolector, preservando precios cuando corresponde.
NumeroOrdenService	Asigna numeros de orden por recolector mediante bloques.
FacturaRecolectorAuditService	Detecta incongruencias en facturas de recolector.
EnterpriseCodeService	Genera, valida y regenera codigo empresarial.
DeviceAccessService	Controla bloqueos por intentos de acceso.
DashboardCacheService	Limpia cache relacionado con dashboard, facturas y produccion.

10. Flujos funcionales

Login y seguridad

1. El usuario entra al sistema.
2. Puede requerirse codigo empresarial.
3. El login valida credenciales y dispositivo.
4. Middleware auth confirma sesion.

5. Middleware activo bloquea usuarios desactivados.
6. Middleware rol limita acceso por rol.

Recolector crea orden

1. Selecciona cliente visible.
2. Selecciona prendas activas del catalogo de recolector.
3. Registra cantidad y color por prenda.
4. El sistema calcula subtotales y total.
5. Se asigna numero de orden.
6. Se guarda `facturas_recolector`.
7. Se guardan lineas en `factura_recolector_detalle`.
8. Se detectan incongruencias si aplica.
9. Opcionalmente se intenta enviar mensaje de WhatsApp Business si esta configurado.

Lavadero registra produccion

1. El sistema sincroniza prendas de lavadero desde el catalogo de recolector.
2. El usuario registra cantidades lavadas por fecha.
3. Se guarda o actualiza producciones.
4. `ProduccionValidationService` compara contra prendas recibidas.
5. El sistema asigna `cantidad_validada`, `total_validado` y `estado_validacion`.
6. Si hay faltantes o sobrantes, crea `incongruencias_produccion`.

Administrador aprueba incongruencia

1. El admin revisa incongruencias pendientes.
2. Si aprueba una incongruencia de produccion, se marca como aprobada.
3. La produccion relacionada pasa a `estado_validacion = aprobado`.
4. Se recalcula la fecha para mantener consistencia.
5. Se limpia cache de produccion.

Factura pagada y comision

1. Recolector o admin marca factura como pagada.
2. Se registra metodo de pago.
3. Se conserva `quincena_origen`.

4. Se asigna quincena_pago segun la quincena activa.
5. Se recalcula pagos_recolector con porcentaje 30%.

Cierre de quincena

1. Admin ejecuta cierre de produccion.
2. Producciones pagables pasan a historial.
3. Reportes por periodo agrupan produccion, facturas, gastos y pagos.

11. Seguridad tecnica

Controles presentes:

- Hash de passwords por Laravel.
- Middleware de autenticacion.
- Middleware de usuario activo.
- Middleware de rol.
- Throttle de login y admin.
- Codigo empresarial.
- Bloqueo de dispositivos por intentos fallidos.
- Registro publico limitado al primer usuario.
- Variables reales fuera de Git.
- APP_DEBUG=false en produccion.
- SSL requerido para PostgreSQL en Render/Supabase.

Puntos de atencion para futuras mejoras:

- Si se exponen tablas por Supabase Data API, habilitar RLS y politicas estrictas.
- Evitar usar service_role en clientes frontend.
- Revisar que cualquier vista SQL futura use seguridad adecuada.
- Mantener backups periodicos de PostgreSQL.

12. Integraciones

Supabase / PostgreSQL

La aplicacion no depende de APIs propias de Supabase para operar; usa Supabase como PostgreSQL administrado.
La conexion llega por DATABASE_URL.

Render

Render ejecuta la app en Docker y usa el endpoint /up como healthcheck.

WhatsApp Business

El código contiene soporte para envío de mensajes por Graph API cuando están configuradas las variables de WhatsApp:

- token
- phone number id
- version de API
- bandera de habilitación

Si no está configurado, la orden se guarda y solo se informa que la automatización no está habilitada.

13. Pruebas y verificación

Comandos usados:

```
composer test
npm run build
php artisan route:list --except-vendor
php artisan config:cache
php artisan route:cache
php artisan view:cache
```

Resultado validado:

- 85 pruebas automatizadas pasadas.
- Build frontend correcto.
- Rutas cargan correctamente.
- Cache de configuración, rutas y vistas correcta.
- /up responde {"success": true}.
- /up/database responde conexión PostgreSQL correcta.

14. Comandos útiles

Instalación local:

```
composer install
npm install
php artisan key:generate
php artisan migrate --force
npm run build
```

Pruebas:

```
composer test
```

Desarrollo:

```
composer run dev
```

Docker local:

```
docker compose up --build
```

Backup dentro del contenedor:

```
php artisan db:backup
```

15. Resumen tecnico para exposicion

Lavanderia Exclusiva es una aplicacion Laravel 12 con PHP 8.2, Blade, Tailwind, Vite y PostgreSQL. Su dominio se centra en controlar recoleccion, clientes, facturas, prendas, produccion, validacion de diferencias, pagos por quincena, gastos y reportes. El despliegue se realiza en Render mediante Docker, con base de datos PostgreSQL externa, normalmente Supabase. La aplicacion esta protegida con autenticacion, roles, usuarios activos, codigo empresarial y bloqueo de dispositivos. La integridad operativa se apoya en migraciones, servicios de dominio y pruebas automatizadas.